MULESOO DIGITAL SOLUTIONS

Step-by-Step Delivery Guide

How to sell & set up the gym system for a new gym — beginner friendly

One master system • Many gyms • No coding experience needed

Generated 26 June 2026

Contents

1. Welcome — what this guide is

This is your plain-English instruction manual for taking the gym system you already own and setting it up for a brand-new gym client. No coding experience is assumed. If you can follow a recipe, you can follow this.

You will do the same short routine for every gym you sell to. The first time will feel slow. By your third gym it will take you about one to two hours from start to finish.

> **The one idea to remember**
> You have ONE master system. For each gym you make a private, isolated COPY of where its data and website live — you never rebuild the system and you never damage your original.

Read sections 1–6 once to understand how it works. Sections 7–16 are the actual steps you repeat for each gym. Section 18 answers the exact question you asked.

2. The big picture: one system, many gyms

Think of your system like a master recipe. Every gym gets a meal cooked from the same recipe, but served on its own plate, with its own name on the menu.

For each gym there are three separate things you set up:

1. A database (Supabase)	A private vault that stores THIS gym's members, payments, bookings. Each gym must have its own — their data never mixes with another gym's.
2. A website (Vercel)	The actual link the gym and its members use. Each gym gets its own website address, built from your one codebase.
3. The settings & branding	The gym's name, colours, plans, prices and contact details — mostly typed into the Admin Settings page, plus one tiny branding edit.

> **Why separate database + website per gym?**
> So Gym A can never see Gym B's members, and if one gym cancels you simply delete its database and website — the others are untouched. This is normal and expected.

3. What changes per gym, what stays the same

Stays the same (your master code — do NOT edit per gym)

- All the features: registration, health screening, payments, classes, check-in, admin dashboard, notifications.
- The way everything works. You improve this once, for everyone.

Changes per gym (set without touching the master)

- Gym name, tagline, contact details, operating hours — typed into Admin Settings.
- Membership plans, prices, add-ons, joining fees — typed into Admin Settings (Catalog).
- Legal text (indemnity, contract, POPIA), gym rules — typed into Admin Settings.
- Owner alert numbers (WhatsApp/Telegram/email) — typed into Admin Settings.
- Payment keys (the gym's own Paystack) — set as environment variables in Vercel.

The one small code touch per gym

The splash/landing page name, tagline, logo and the main accent colour are not yet read from Settings. You change these in a SAFE per-gym branch (Step 4–5), so your master stays clean. This is the only code edit, and it is copy-paste simple.

4. What DELIVERY.md is and how it helps you

DELIVERY.md is a short text file already inside your project. It is your technical cheat-sheet — the quick reference version of this guide. Open it any time in your code editor or on GitHub.

What it contains

- The exact deploy steps for Vercel and the list of environment variables to set.
- How the scheduled jobs (crons) run, and how to trigger them manually.
- A ready-made end-to-end TEST checklist (registration, member portal, admin).
- A POPIA compliance summary and a security summary you can show a client.
- A "known follow-ups" list so you always know what is and isn't built.

> **How to use it**
> This PDF teaches you the WHY and the full routine. DELIVERY.md is the fast WHAT — keep it open in a second window while you work through a gym, and tick off its test checklist before every handover.

There is also db/README.md (how to create the database) and RECOVERY.md (how to restore everything if a computer is lost). Same idea: short, practical reference files.

5. Accounts & tools you need (set up once)

Before your first gym, get these ready. Most are free to start.

GitHub account	Holds your master code. You already have this (Ethan5322/yoyogym).
Vercel account	Builds and hosts each gym's website. Free tier is fine to start.
Supabase account	Hosts each gym's database. Free tier covers a small gym.
A code editor	VS Code (free). Used only for the tiny branding edit.
Node.js installed	Lets you run the seed commands. Install the LTS version.
Git installed	Lets you make a safe per-gym branch.

For EACH gym you will also collect (from the gym owner)

- Their Paystack account API keys (so payments go to THEM, not you).
- Their Brevo email sender (for member emails) and owner WhatsApp number.
- Their logo image, brand colour, plans & prices, address and contact details.

> **Tip**
> Make a simple Google Form with all of the above. Send it to every new gym so you never chase missing details.

6. Your "master" — keep the original safe

Your master is the "main" branch of your GitHub repo. Treat it like the original master tape: you copy from it, you do not scribble on it.

The golden rules

- Never make gym-specific edits directly on "main". Main stays generic and clean.
- For each gym, create a BRANCH (a safe parallel copy) named after the gym. You edit branding there.
- Each gym's Vercel website is connected to that gym's branch, so its branding only affects that gym.
- When you improve the system, you do it on main, then merge main into each gym's branch to update them.

> **Why a branch and not a full copy?**
> A branch keeps every gym linked to your one master, so improvements flow to all of them. A full separate copy would force you to update each gym by hand forever.

Don't worry if "branch" and "merge" are new words — Step 4 shows the exact clicks/commands, and the Glossary (section 20) explains every term.

7. STEP 1 — Collect the gym's information

1 Gather everything before you build

Starting with all details in hand makes the rest fast. Collect:

- Gym name, tagline, logo file, brand colour (a hex code like #1E88E5).
- Address, phone, email, website, operating hours.
- Membership plans with prices, joining fees, add-ons (or use sensible defaults and adjust later).
- Their Paystack public & secret keys (from the gym's Paystack dashboard).
- Their Brevo API key + sender email; the owner's WhatsApp number (+27…).
- Who the owner login should belong to (their name + email).

> **No Paystack yet?**
> You can still deliver — online card payments simply stay off until they add keys. Staff can record cash/EFT payments in the admin in the meantime.

8. STEP 2 — Create the gym's database (Supabase)

2 Make a private vault for this gym's data

1. Go to supabase.com and log in. Click "New Project".
2. Name it clearly, e.g. "powerfit-gym". Choose a region close to South Africa.
3. Set a strong database password and save it in your password manager.
4. Wait ~2 minutes for the project to finish setting up.
5. Open Project Settings !' API. Copy two things: the Project URL and th

> **Keep the service_role key secret**
> It has full access to the database. It only ever goes into Vercel's environment variables (Step 7) — never into the code, never shared in chat or email.

Each gym gets its own brand-new Supabase project. That is what keeps their members' data completely separate from every other gym.

9. STEP 3 — Load the schema & starter data

3 Create the tables, then add starter plans

3a. Create the tables

1. In Supabase, open the "SQL Editor" (left menu).
2. In your project, open the file db/schema.sql and copy ALL of it.
3. Paste it into the SQL Editor and click "Run". You should see success, no errors.

This builds all the tables (members, payments, classes, etc.) in one go. db/README.md explains this too.

3b. Add the owner login & starter catalogue

On your computer, in the project folder, create a temporary .env file pointing at THIS gym's Supabase (URL + service_role key + a JWT secret), then run:

1. npm run seed:owner — creates the gym's owner admin login.
2. npm run seed:catalog — adds default membership plans & add-ons.

> **Heads up**
> The owner username/email/password come from the SEED_OWNER_* lines in your .env. Set them to the gym owner's details (or a temporary password they change later).

10. STEP 4 — Make a safe branch for this gym

4 Copy the master so you can brand safely

This protects your original. In your project folder, open a terminal and run:

1. git checkout main — make sure you start from the clean master.
2. git pull — get the latest master.
3. git checkout -b gym-powerfit — create + switch to this gym's branch.

You are now on the gym's branch. Anything you change here stays here and never touches "main".

> **Naming tip**
> Use "gym-" plus the gym's name, e.g. gym-powerfit, gym-ironworks. You'll connect this exact branch to the gym's Vercel site in Step 6.

If terminals feel scary, VS Code has a "Source Control" panel with buttons for the same actions — and the Glossary explains each term.

11. STEP 5 — Brand the splash (name, tagline, logo, colour)

5 The only code edit — copy/paste simple

On the gym's branch, open src/pages/Splash.jsx. Change just the text:

- Replace "Your Gym Name" with the gym's name.
- Replace "Train harder. Live stronger." with the gym's tagline.
- For the logo: put their logo file in the /public folder (e.g. logo.png) and swap the placeholder box for an image — or simply change the letter "G" to their initial for now.

Set the accent colour

Open src/index.css and find the line that sets --accent (around the top). Change the hex value to the gym's brand colour, e.g. --accent: #1E88E5; for blue.

> **Save your work to the branch**
> After editing, run: git add -A then git commit -m "Brand for PowerFit" then git push -u origin gym-powerfit. This saves the branding onto the gym's branch only.

That's the entire code touch. Everything else (plans, prices, legal, contacts) is done by typing in Admin Settings in Step 8 — no code.

12. STEP 6 — Create the gym's website (Vercel)

6 Publish this gym's own website

1. Go to vercel.com → "Add New… → Project".
2. Import your GitHub repo (the same yoyogym repo every time).
3. Under "Git Branch", choose this gym's branch (e.g. gym-powerfit), NOT main.
4. Framework preset: Vite. The build settings come from your vercel.json automatically.
5. Before deploying, add the environment variables (Step 7), then click Deploy.

> **One repo, many websites**
> You import the SAME repo for every gym; the difference is the branch you pick and the environment variables you set. That's how one codebase powers many independent gym websites.

After deploying, give the gym a friendly address — a subdomain like powerfit.yourbrand.co.za, or connect their own domain in Vercel → Settings → Domains.

13. STEP 7 — Add the environment variables

7 Connect this website to this gym's services

In the Vercel project's Settings' Environment Variables, add the values checklist). The important ones:

SUPABASE_URL	This gym's Supabase Project URL (Step 2).
SUPABASE_SERVICE_ROLE_KEY	This gym's service_role key (Step 2). Secret.
SUPABASE_SCHEMA	gym
JWT_SECRET	A long random string (unique per gym).
PAYSTACK_SECRET_KEY / PUBLIC_KEY	The gym's own Paystack keys.
BREVO_API_KEY / SENDER	For member emails.
CALLMEBOT / OWNER_EMAIL	Owner alert contacts.
CRON_SECRET	A random string to protect scheduled jobs.

Then redeploy
After adding variables, trigger a redeploy so they take effect. Confirm success by visiting https://THEIR-SITE/api/health — you should see {"status":"ok"}.

14. STEP 8 — Configure everything in Admin Settings

8 Type the gym's details — no code

Open https://THEIR-SITE/admin/login and sign in with the owner account you seeded. Go to Settings and fill in each section, clicking Save on each:

- Gym Profile — name, tagline, accent colour, phone, email, address, hours, welcome message.
- Owner Notifications — WhatsApp/Telegram/email for instant alerts.
- Contract Discounts — discount % for longer contracts.
- Access & Compliance — capacity, session length, peak hours, plan rules.
- Gym Rules, Indemnity, Membership Contract, POPIA — review the defaults, edit, Save.

Then open the Catalog page to adjust membership plans, prices, joining fees and add-ons to match what the gym sells.

> **Good news**
> The gym NAME you set here flows automatically into the registration flow, member emails and the PDF membership cards. Most of the branding is genuinely no-code.

15. STEP 9 — QR codes, handover & training

9 Give the gym what they actually use

In the admin, open QR Codes and download the two print-ready codes:

- New Member QR ’ opens registration (/register).
- Existing Member QR ’ opens the member portal (/member).

Hand the gym owner their "delivery pack":

The website link	Their live address — this IS the product (no app to install).
Admin login	Username + password, shared securely (not plain email).
QR code files	Print for the door, reception desk, flyers, social media.
A short how-to	One page: how to verify members, add classes, see payments.

> **Train them live**
> A 30-minute call walking the owner through Verify, Members, Classes and Payments dramatically reduces support questions later.

16. STEP 10 — Test before you hand over

10 Tick these off (also in DELIVERY.md)

- Open /register — the splash/flow shows the gym's name and a time-of-day greeting.
- Complete a test registration: personal info, PAR-Q, plan, agreement, success screen.
- Download the membership PDF — it shows the gym name and a QR code.
- If Paystack is set: do a test payment and confirm the member becomes active.
- Log into /admin — Dashboard loads; Verify a code shows a green "access granted".
- Members, Classes, Payments, Settings all open and save.
- Visit /api/health ’ {"status":"ok"}.

> **If something fails**
> Check the environment variables first (90% of issues), then re-run the relevant step. Section 19 has a troubleshooting list.

17. Selling to your 2nd, 3rd, 10th gym (the loop)

Every new gym is the SAME routine. You do not start over and you do not rebuild anything:

1. Collect details (Step 1).
2. New Supabase project + run schema + seed (Steps 2–3).
3. New branch off main, named for the gym (Step 4).
4. Edit the splash name/tagline/logo + accent on that branch (Step 5).
5. New Vercel project from the same repo, pointed at the gym's branch (Step 6).
6. Add that gym's environment variables (Step 7).
7. Configure Settings + Catalog in the admin (Step 8).
8. QR codes, handover, train, test (Steps 9–10).

> **It gets fast**
> By your third gym this is a ~1–2 hour checklist. Print Steps 7–16 and tick them off each time.

18. Your exact question, answered plainly

You asked: "When I sell to another gym, do I need to ask Claude to change the name/logo of the original file, and create the SQL and Vercel project again? How do I do this for a different gym?"

Do I need to ask Claude each time? — No.

Everything is a repeatable checklist you do yourself (Steps 1–10). You only need a developer/Claude for NEW features or fixes to the master — not for setting up a gym.

Do I edit the ORIGINAL Yoyo GYM files? — No.

You never edit "main". You make a per-gym branch (Step 4) and do the tiny branding edit there, so your original stays clean and reusable forever.

Do I create a new database and Vercel project each time? — Yes.

Each gym needs its own Supabase database and its own Vercel website so their data and site are separate. This is normal, expected, and takes ~30 minutes once you're used to it.

How is the name/logo/colour actually changed?

- Name, plans, prices, legal, contacts, hours ⁉ typed into Admin Setting
- Splash name/tagline/logo + accent colour ⁉ one small edit on the gym'

> **Want it to be 100% no-code?**
> There is an optional one-time upgrade that makes the splash, logo and accent colour load from Admin Settings too. After that, a new gym is purely: new database + new website + fill in Settings. Ask for the "dynamic branding" upgrade when you're ready.

19. Updating all gyms + troubleshooting

Pushing an improvement to every gym

1. Make the change on "main" and push it to GitHub.
2. For each gym branch: merge main into it (git checkout gym-x, then git merge main, then git push).
3. Vercel redeploys that gym automatically. Repeat per gym.

> **Even easier later**
> Once you take the optional dynamic-branding upgrade, most gyms can run straight off "main" with no per-gym branch — then a single push updates everyone at once.

Common problems & quick fixes

Site loads but data errors	Check SUPABASE_URL / service_role key in Vercel; redeploy.
Can't log into admin	Re-run npm run seed:owner against THIS gym's database.
Payments not working	Paystack keys missing/incorrect in Vercel env vars.
Emails not sending	Set BREVO_API_KEY and a verified sender email.
/api/health not ok	A required env var is missing — compare with .env.example.

20. Glossary of words you'll see

Repository (repo)	The folder of your code stored on GitHub.
Branch	A safe parallel copy of the code where you can make changes without affecting the master ("main").
main	Your master branch — the clean original.
Merge	Bringing changes from one branch into another (e.g. main!' a
Commit	Saving a snapshot of your changes with a short message.
Push	Uploading your commits to GitHub.
Deploy	Publishing the website so people can use it (Vercel does this).
Environment variable	A secret setting (keys, URLs) stored in Vercel, not in the code.
Supabase	The database service — one project per gym.
Schema	The structure of the database (the tables) — created by db/schema.sql.
Seed	Adding starter data (owner login, default plans).
Service role key	A powerful secret database key — keep it private.
Subdomain	A branded address like powerfit.yourbrand.co.za.
Admin Settings	The no-code page where you type each gym's details.

> **You've got this**
> Follow the steps in order, keep DELIVERY.md open beside you, and tick off the Step 10 test list before every handover. Each gym gets faster.